



Évaluation Finale — Module React Native M1

Développement d'une application mobile en React Native / Expo

Durée : 2 mois (travail personnel) **Livrable** : Dépôt GitHub + APK / lien Expo Go + Document individuel + Soutenance **Note** : /20 **Travail** : Équipe de 2 à 3 personnes — solo accepté



Contexte du projet

Vous êtes une petite équipe de développeurs mobiles freelance. Un client fictif vous confie la création d'une **application mobile** de A à Z. Vous choisissez votre sujet parmi la liste ci-dessous et vous répartissez le travail au sein de l'équipe.

L'objectif de cette évaluation est de démontrer votre maîtrise de **React Native avec Expo**, de la conception à la livraison, en passant par l'intégration d'API, la persistance des données, la navigation et la qualité du code.

Note sur le travail solo : Un étudiant travaillant seul n'est pas pénalisé sur la note, mais le volume de fonctionnalités attendu reste le même. Le document individuel sera remplacé par un document de réflexion globale sur le projet.



Exigences Techniques Communes

Ces critères sont **obligatoires pour tous les sujets**. Ils constituent le socle technique évalué indépendamment du sujet choisi.

Stack imposée

- **Framework** : React Native avec **Expo** (SDK 51+)
- **Langage** : JavaScript ou TypeScript (*TypeScript fortement conseillé*)
- **Navigation** : React Navigation v6+ (Stack + Tab au minimum)
- **Persistance** : Supabase (base de données + auth) et/ou AsyncStorage selon le besoin
- **API externe** : Au moins **une API REST publique** consommée avec fetch ou axios
- **Styling** : StyleSheet natif ou NativeWind (*pas d'UI kit imposé*)

Structure du code

- Dossier `src/` organisé : `components/` , `screens/` , `hooks/` , `services/` , `utils/`
- Composants réutilisables (`Button` , `Card` , `Input` ...) isolés dans `components/`
- Les appels Supabase et API regroupés dans `services/` , pas dispersés dans les écrans
- Pas de `console.log` laissés en production

Qualité UX

- Feedback utilisateur systématique : `ActivityIndicator` pendant les chargements, messages d'erreur lisibles
- Application fonctionnelle sur **iOS et Android** (testé via Expo Go ou émulateur)
- Design responsive (pas de texte coupé, pas d'éléments qui débordent)
- Gestion du cas liste vide (empty state) sur chaque liste affichée

Les Sujets

Choisissez **un seul sujet par équipe**. Ce choix est définitif après validation par votre formateur dans les **10 premiers jours**. Deux équipes peuvent choisir le même sujet.

Sujet 1 — CineMatch

L'app qui met fin aux débats interminables pour choisir un film

CONTEXTE CLIENT

Votre client veut créer une application pour aider des groupes d'amis à choisir un film à regarder ensemble. Chaque membre swipe des films en secret, et l'app révèle ceux qui ont plu à tout le monde.

API IMPOSÉE

The Movie Database (TMDB) — developers.themoviedb.org Clé API gratuite à obtenir via inscription. Endpoints principaux : `/movie/popular` , `/movie/{id}` , `/search/movie` .

FONCTIONNALITÉS OBLIGATOIRES

- ☒ **Authentification** : Inscription et connexion via Supabase Auth (email + mot de passe)
- ☐ **Écran de swipe** : Affichage des films un par un (poster, titre, année, note TMDB) avec gestion du geste swipe gauche (refus) / droite (like) via `PanResponder` ou `react-native-gesture-handler`
- ☐ **Écran de match** : Affichage des films likés par l'utilisateur, triés par note TMDB — likes persistés dans Supabase

- ☐ **Fiche film** : Synopsis, durée, genres, note, lien bande-annonce YouTube, casting des 5 premiers acteurs
- ☒ **Recherche** : Barre de recherche pour trouver un film précis avec debounce implémenté
- ☐ **Historique** : Les films swipés (likés et refusés) sont sauvegardés dans Supabase et consultables dans un écran dédié

FONCTIONNALITÉS BONUS (+1 PT CHACUNE, MAX +2)

- Système de filtres par genre avant de commencer à swiper
- Animation fluide de la carte avec rotation pendant le swipe (Reanimated 2)

LIVRABLES SPÉCIFIQUES

README indiquant comment obtenir et configurer la clé API TMDb via un fichier `.env`.

Sujet 2 — WanderLog

Le carnet de voyage de l'ère numérique

CONTEXTE CLIENT

Votre client veut une application pour documenter ses aventures : photos prises sur le vif, notes de voyage, carte des lieux visités, le tout synchronisé sur son compte.

API IMPOSÉE

Open-Meteo — open-meteo.com (météo gratuite, sans clé API) Afficher la météo actuelle du lieu sélectionné sur la carte.

FONCTIONNALITÉS OBLIGATOIRES

- ☐ **Authentification** : Inscription et connexion via Supabase Auth (email + mot de passe)
- ☐ **Liste des voyages** : Affichage de tous les voyages de l'utilisateur connecté (nom, destination, dates, photo de couverture) — données stockées dans Supabase
- ☐ **Création de voyage** : Formulaire avec nom, destination, dates de début/fin
- ☐ **Journal de voyage** : Dans chaque voyage, ajout d'entrées avec titre, texte, date et photo prise depuis l'appareil (`expo-image-picker`) — photos uploadées dans Supabase Storage
- ☐ **Carte interactive** : Visualisation des lieux épinglés via `react-native-maps` — l'utilisateur peut épingler sa position actuelle (`expo-location`)
- ☐ **Météo du lieu** : Appel Open-Meteo pour afficher la météo actuelle quand on consulte un lieu épinglé

FONCTIONNALITÉS BONUS (+1 PT CHACUNE, MAX +2)

- Export PDF du carnet de voyage (`expo-print` + `expo-sharing`)
- Partage de voyage : rendre un carnet public et consultable via un lien

👊 Sujet 3 — GymBuddy

Ton coach personnel dans la poche

CONTEXTE CLIENT

Votre client veut une app de suivi d'entraînement pour sportifs autodidactes : créer ses programmes, logger ses séances, visualiser ses progrès — tout en gardant son historique synchronisé.

API IMPOSÉE

wger Workout Manager — wger.de/api/v2 API open source, sans clé. Endpoints : `/exercise/`, `/exercisecategory/`, `/muscle/`.

FONCTIONNALITÉS OBLIGATOIRES

- ☐ **Authentification** : Inscription et connexion via Supabase Auth (email + mot de passe)
- ☐ **Bibliothèque d'exercices** : Liste paginée depuis l'API wger, filtrable par catégorie musculaire, avec barre de recherche
- ☐ **Création de programme** : L'utilisateur crée un programme nommé et y ajoute des exercices avec séries, répétitions et poids cible — sauvegardé dans Supabase
- ☐ **Log de séance** : L'utilisateur démarre une séance depuis un programme et log les séries effectuées (poids réel + reps réelles), avec timer de repos entre les séries
- ☐ **Historique** : Liste des séances passées consultables avec le détail des séries — données dans Supabase
- ☐ **Graphiques de progression** : Courbe du poids soulevé sur un exercice donné dans le temps (`victory-native` ou `react-native-gifted-charts`)

FONCTIONNALITÉS BONUS (+1 PT CHACUNE, MAX +2)

- Calcul automatique du 1RM (répétition maximale) via la formule d'Epley
- Notifications locales pour rappeler de faire sa séance (`expo-notifications`)

🎵 Sujet 4 — VibeCheck

Découvre ta musique selon ton humeur du moment

CONTEXTE CLIENT

Votre client veut une app musicale originale : l'utilisateur choisit son humeur, l'app génère une playlist personnalisée et conserve l'historique de ses humeurs.

API IMPOSÉE

Deezer API — developers.deezer.com Gratuite, sans OAuth pour la partie recherche et exploration. Endpoints : `/search`, `/chart`, `/genre`, `/radio`.

FONCTIONNALITÉS OBLIGATOIRES

- ☐ **Authentification** : Inscription et connexion via Supabase Auth (email + mot de passe)
- ☐ **Sélecteur d'humeur** : Interface visuelle et animée pour choisir parmi 6 humeurs (Joyeux, Nostalgique, Énergique, Mélancolique, Concentré, Festif) — chaque humeur mappe vers des genres musicaux
- ☐ **Génération de playlist** : Appel Deezer selon les genres de l'humeur, affichage d'une liste de 20 titres avec artiste, album et pochette
- ☐ **Lecteur audio intégré** : Lecture des previews 30 secondes ([expo-av](#)) avec barre de progression, boutons play/pause/suivant/précédent
- ☐ **Favoris** : L'utilisateur peut sauvegarder des tracks en favoris dans Supabase, consultables dans un écran dédié
- ☐ **Historique d'humeurs** : Les humeurs choisies sont enregistrées dans Supabase — affichage d'un graphique ou calendrier des 30 derniers jours

FONCTIONNALITÉS BONUS (+1 PT CHACUNE, MAX +2)

- Animations Reanimated 2 sur le sélecteur d'humeur (morphing de couleurs selon l'humeur)
- Partage de la playlist générée via [expo-sharing](#)

Sujet 5 — BrainDuel

Le choc des cerveaux — quiz multijoueur

CONTEXTE CLIENT

Votre client veut un jeu de quiz engageant avec des scores persistants, des catégories variées, un chronomètre et un leaderboard partagé entre tous les joueurs.

API IMPOSÉE

Open Trivia Database — opentdb.com/api_config.php Entièrement gratuite, sans clé. Plus de 5 000 questions, multiples catégories et difficultés.

FONCTIONNALITÉS OBLIGATOIRES

- ☐ **Authentification** : Inscription et connexion via Supabase Auth (email + mot de passe) — le pseudo est saisi à l'inscription
- ☐ **Écran de configuration** : Choix de la catégorie (liste récupérée depuis l'API), difficulté (easy/medium/hard), nombre de questions (10/20/30)
- ☐ **Mode Solo** : Partie contre le temps (chronomètre dégressif par question), score calculé selon rapidité et exactitude
- ☐ **Écran de résultat** : Score final, récapitulatif des bonnes et mauvaises réponses, animation victoire/défaite
- ☐ **Leaderboard global** : Top 10 des meilleurs scores persisté dans Supabase, visible par tous les utilisateurs avec pseudo et date

- ☐ **Décodage HTML** : Les questions OTDB contiennent des entités HTML (ex: `&`) — les décoder correctement avant affichage

FONCTIONNALITÉS BONUS (+1 PT CHACUNE, MAX +2)

- Mode Duel local : deux joueurs sur le même appareil s'affrontent en tour par tour avec masque entre les tours
- Sons et vibrations (`expo-haptics` + `expo-av`) sur bonne/mauvaise réponse

Sujet 6 — ZenFlow

Ton rituel bien-être quotidien

CONTEXTE CLIENT

Votre client veut une app de bien-être douce et épurée : méditations guidées, exercices de respiration, suivi des habitudes et rappels bienveillants — synchronisés sur le compte utilisateur.

API IMPOSÉE

Quotable API — api.quotable.io Gratuite, sans clé. Endpoint : `/random?tags=inspirational` pour afficher une citation inspirante chaque jour.

FONCTIONNALITÉS OBLIGATOIRES

- ☐ **Authentification** : Inscription et connexion via Supabase Auth (email + mot de passe)
- ☐ **Tableau de bord** : Citation du jour (API Quotable), streak de jours consécutifs actifs, accès rapide aux modules
- ☐ **Module Méditation** : Liste de sessions (5, 10, 20 min) avec description, lecteur audio (`expo-av`) avec son d'ambiance (fichiers audio locaux), minuteur animé avec cercle de progression
- ☐ **Module Respiration** : Animations de cercle guidant un cycle de respiration paramétrable (ex: 4-4-4-4 — inspirer / retenir / expirer / retenir)
- ☐ **Tracker d'habitudes** : L'utilisateur crée des habitudes quotidiennes, les coche chaque jour — données persistées dans Supabase — visualisation de la constance sur 30 jours (grille de cases colorées type GitHub contributions)
- ☐ **Rappels** : Notifications locales programmables par habitude (`expo-notifications`)

FONCTIONNALITÉS BONUS (+1 PT CHACUNE, MAX +2)

- Thème sombre/clair avec switch manuel et respect du thème système (`useColorScheme`)
- Statistiques hebdomadaires envoyées par email via une Edge Function Supabase

Sujet 7 — FridgeChef

Transforme tes restes en chef-d'œuvre culinaire

CONTEXTE CLIENT

Votre client veut une app qui aide à cuisiner avec ce qu'on a déjà chez soi : on entre ses ingrédients, l'app propose des recettes réalisables et mémorise les favoris sur le compte utilisateur.

API IMPOSÉE

TheMealDB — themealdb.com/api.php Entièrement gratuite, sans clé. Endpoints : [/search.php](#) , [/filter.php](#) , [/lookup.php](#) , [/categories.php](#) .

FONCTIONNALITÉS OBLIGATOIRES

- ☐ **Authentification** : Inscription et connexion via Supabase Auth (email + mot de passe)
- ☐ **Frigo virtuel** : Écran listant les ingrédients disponibles avec ajout, modification et suppression — données persistées dans Supabase
- ☐ **Suggestions de recettes** : Basées sur les ingrédients du frigo (requête par ingrédient principal), affichage en grille avec photo, nom et catégorie
- ☐ **Fiche recette complète** : Photo, catégorie, origine, liste des ingrédients avec quantités, instructions étape par étape, lien YouTube si disponible
- ☐ **Mode cuisine** : Affichage plein écran étape par étape avec boutons suivant/précédent — écran qui ne se met pas en veille ([expo-keep-awake](#))
- ☐ **Favoris** : Recettes favorites sauvegardées dans Supabase, consultables hors-ligne via un cache local

FONCTIONNALITÉS BONUS (+1 PT CHACUNE, MAX +2)

- Scanner de code-barres produit ([expo-barcode-scanner](#)) pour ajouter un ingrédient au frigo par scan
- Liste de courses générée automatiquement à partir des ingrédients manquants d'une recette, avec cases à cocher

Livrables & Instructions de Rendu

Ce que l'équipe doit rendre

1. Dépôt GitHub

- Repo **privé** nommé selon le format indiqué ci-dessous
- Accès donné à l'adresse GitHub du formateur **avant la deadline**
- Commits réguliers et explicites — l'historique doit montrer les contributions des différents membres

- Fichier `README.md` à la racine contenant :
 - Sujet choisi et description courte
 - Composition de l'équipe (prénom, nom, partie traitée)
 - Instructions d'installation (`npm install` , `npx expo start`)
 - Clé(s) API et variables Supabase nécessaires — fichier `.env.example` fourni
 - Librairies utilisées et justification des choix

2. Application fonctionnelle

Au choix :

- **Lien Expo Go** : QR Code fonctionnel fourni dans le README (projet publié sur Expo)
- **APK Android** : Fichier `.apk` généré via `eas build` et joint dans le rendu

L'app doit démarrer sans erreur avec `npx expo start` sur la machine du correcteur.

3. Document individuel *(rendu par chaque membre de l'équipe — y compris en solo)*

Chaque membre rend un **document PDF de 1 à 2 pages** contenant :

Partie A — Ma contribution au projet

- Les écrans ou fonctionnalités que j'ai développés
- Les difficultés techniques rencontrées et comment je les ai résolues
- Ce que j'aurais fait différemment avec plus de temps

Partie B — Mon utilisation de l'IA

- Les outils d'IA utilisés (ChatGPT, Copilot, Claude, Cursor...)
- Des exemples concrets de prompts utilisés et ce qu'ils ont produit
- Ce que l'IA a bien fait, ce qu'elle a mal fait
- Ma réflexion personnelle : en quoi l'IA a-t-elle changé ma façon de coder sur ce projet ?

⚠ Ce document est **individuel et obligatoire**, même en équipe. Il compte dans la note finale. Une réponse générique du type "j'ai utilisé ChatGPT pour m'aider" sans exemple concret ne sera pas acceptée.

4. Soutenance

Présentation de **20 minutes par équipe** en présentiel ou distanciel :

- 10 min — démo live de l'application (scénario utilisateur complet)
- 7 min — revue de code (chaque membre explique sa partie)
- 3 min — questions

En solo : 15 minutes (10 min démo + 5 min questions).

Nommage du repo GitHub

NOM1-NOM2-NOM3_M1RN_NomDuSujet

Exemples :

- DUPONT-MARTIN_M1RN_CineMatch
- BERNARD-LEROY-SIMON_M1RN_GymBuddy
- MOREAU_M1RN_ZenFlow (solo)

Nommage du document individuel

NOM_Prenom_M1RN_DocumentIndividuel.pdf

Exemple : DUPONT_Marie_M1RN_DocumentIndividuel.pdf

Dépôt & partage

Envoyez par email à votre formateur **avant la deadline** :

1. Le lien vers votre repo GitHub (accès donné au préalable)
2. Le lien Expo Go ou le fichier APK
3. Les documents individuels de chaque membre (un PDF par personne)
4. Vos disponibilités pour la soutenance

⚠ Un repo non partagé ou sans accès = rendu non reçu = 0 pour toute l'équipe. ⚠ Un repo avec un seul commit la veille de la deadline = -3 pts (absence de travail itératif). ⚠ Document individuel absent = -3 pts sur la note personnelle du membre concerné.

Grille de Notation

La note est composée de **deux parties** : une note d'équipe et une note individuelle.

Partie Équipe (/14 pts)

#	Critère	Barème	Points obtenus
1	Navigation (Stack + Tab fonctionnels, transitions propres, gestion du bouton retour)	/2	
2	Authentification Supabase (inscription, connexion, déconnexion, données liées à l'utilisateur)	/2	
3	Consommation API externe (appel correct, gestion du loading, gestion des erreurs réseau)	/2	
4	Persistance Supabase (données sauvegardées et rechargées correctement entre sessions)	/1,5	
5	Qualité UI/UX (cohérence visuelle, responsive, feedbacks, empty states, pas de bug visuel)	/2	
6	Qualité du code (structure src/, composants réutilisables, appels API dans services/)	/2	
7	README et configuration (installation claire, .env.example, composition de l'équipe)	/1	
8	Historique Git (commits réguliers des différents membres, messages explicites)	/1,5	
Sous-total équipe		/14	

Fonctionnalités du sujet (/4 pts)

Les 6 fonctionnalités obligatoires sont notées **collectivement**.

Fonctionnalité obligatoire	Complète	Partielle	Absente
Fonctionnalité 1 (Auth)	0,5 pt	0,25 pt	0 pt
Fonctionnalité 2	0,5 pt	0,25 pt	0 pt
Fonctionnalité 3	0,5 pt	0,25 pt	0 pt
Fonctionnalité 4	0,5 pt	0,25 pt	0 pt
Fonctionnalité 5	0,5 pt	0,25 pt	0 pt
Fonctionnalité 6	0,5 pt	0,25 pt	0 pt

Fonctionnalité 7	0,5 pt	0,25 pt	0 pt
Fonctionnalité 8	0,5 pt	0,25 pt	0 pt
Sous-total fonctionnalités			/4

Partie Individuelle (/2 pts)

Chaque membre est noté **individuellement** sur son document.

Critère	Barème
Contribution clairement identifiée (écrans ou features décrits avec précision)	/0,5
Difficultés et solutions documentées (analyse technique honnête et concrète)	/0,5
Utilisation de l'IA documentée (exemples de prompts, résultats, analyse critique)	/0,75
Réflexion personnelle (ce que ça a changé dans sa façon de travailler)	/0,25
Sous-total document individuel	/2

Bonus (max +2 pts, non plafonnés au-dessus de 20)

Bonus	Points
Fonctionnalité bonus 1 du sujet (fonctionnelle et intégrée proprement)	+1 pt
Fonctionnalité bonus 2 du sujet (fonctionnelle et intégrée proprement)	+1 pt

Pénalités automatiques

Situation	Pénalité
Repo GitHub non partagé ou inaccessible avant la deadline	Rendu non reçu — 0 pour l'équipe
Historique Git : unique commit à la dernière minute	-3 pts (équipe)
Application qui plante au démarrage (crash immédiat)	-4 pts (équipe)
Clé API ou URL Supabase hardcodée dans le code source	-2 pts (équipe)
Document individuel absent	-3 pts (membre concerné uniquement)

Absence à la soutenance sans justification préalable	-4 pts (membre concerné uniquement)
Code manifestement copié depuis un projet existant sans adaptation	Signalement académique

Récapitulatif

Partie	Barème
Critères techniques équipe	/14
Fonctionnalités du sujet	/4
Document individuel	/2
Bonus	+2 max
TOTAL	/20



Conseils pour réussir



Organisation de l'équipe

- **Décidez la répartition dès la semaine 1** : ne commencez pas tous sur le même écran
- Utilisez les **branches Git** : une branche par feature, merge dans `main` quand c'est stable
- Faites des **points d'équipe hebdomadaires** courts (15 min) pour synchroniser l'avancement
- L'auth Supabase est le **socle de tout** — faites-la en premier ensemble avant de vous répartir



Planning conseillé sur 8 semaines

Semaine	Objectif
1	Mise en place du projet, auth Supabase fonctionnelle, navigation de base — ensemble
2	Répartition des tâches, premiers écrans, connexion à l'API externe
3 - 4	Développement des fonctionnalités principales, intégration Supabase par feature
5 - 6	Gestion des erreurs, cas limites, liaison entre les parties de l'équipe
7	Finitions UI, tests sur device réel, bonus si le temps le permet

8

Polish final, README, documents individuels, préparation de la soutenance

Règle d'or : Une app avec 4 fonctionnalités qui marchent parfaitement vaut mieux qu'une app avec 6 fonctionnalités qui buggent.

Outils recommandés

Besoin	Outil conseillé
Démarrage projet	<code>npx create-expo-app</code>
Test sur mobile	Expo Go (iOS et Android)
Base de données + Auth	Supabase (dashboard + <code>@supabase/supabase-js</code>)
Icônes	<code>@expo/vector-icons</code> (inclus dans Expo)
Variables d'environnement	fichier <code>.env</code> + <code>expo-constants</code>
Requêtes HTTP	<code>axios</code> ou <code>fetch</code> natif
Manipulation de dates	<code>date-fns</code>
Formulaires	<code>react-hook-form</code>
Animations avancées	<code>react-native-reanimated</code> (déjà inclus dans Expo)
Collaboration	Git branches + pull requests entre membres

Conseils sur l'utilisation de l'IA

L'IA est **autorisée et même encouragée** — le marché du travail l'utilise, vous devez apprendre à le faire intelligemment. Ce qui est évalué dans le document individuel, c'est votre **recul critique** sur cet usage.

Quelques bonnes pratiques :

- Utilisez l'IA pour débloquer des problèmes précis, pas pour générer des écrans entiers que vous ne comprenez pas
- Relisez et comprenez **tout** le code que l'IA vous produit avant de l'intégrer — la soutenance vous demandera de l'expliquer
- Notez vos prompts au fur et à mesure : vous en aurez besoin pour le document individuel
- Comparez les résultats de l'IA avec la documentation officielle — elle se trompe parfois

⚠ Erreurs classiques à éviter

- Commencer par le design avant d'avoir l'auth Supabase et l'API qui fonctionnent
- Mettre les clés Supabase ou API directement dans le code source pushé sur GitHub (*utilisez un fichier `.env` listé dans `.gitignore`*)
- Ne pas gérer le cas où l'API ou Supabase retourne une erreur — l'app ne doit jamais bloquer silencieusement
- Oublier de tester sur un vrai appareil — les émulateurs ne reproduisent pas tous les comportements
- Attendre la dernière semaine pour commencer (*l'historique Git est noté*)
- Rédiger le document individuel la veille — il demande de la réflexion, pas du remplissage

? FAQ Évaluation

Q : On est 3 dans l'équipe, est-ce qu'on a plus de fonctionnalités à faire ? Non. Le cahier des charges est le même quelle que soit la taille de l'équipe. Une équipe de 3 doit simplement produire un **code de meilleure qualité** (tests, animations, polish UI) et peut viser les bonus plus sereinement.

Q : Peut-on utiliser TypeScript ? Oui, et c'est fortement encouragé. La lisibilité et la maintenabilité TypeScript sont valorisées dans le critère qualité du code.

Q : Peut-on utiliser une UI library (React Native Paper, NativeBase, Tamagui...) ? Oui, à condition que vous maîtrisiez les composants utilisés. La soutenance peut vous demander d'expliquer n'importe quelle partie du code.

Q : L'API imposée est hors service ou a changé, que faire ? Contactez le formateur immédiatement. Une alternative sera proposée. Ne changez pas d'API de votre propre chef sans validation.

Q : Peut-on utiliser les Edge Functions Supabase ou d'autres fonctionnalités avancées ? Oui, pour les fonctionnalités bonus. Les fonctionnalités de base doivent fonctionner sans dépendance à des services non fiables.

Q : Mon app tourne sur iOS mais plante sur Android (ou l'inverse), est-ce grave ? L'app doit fonctionner sur les deux plateformes. Un bug bloquant sur l'une d'elles impacte la note de qualité UX. Testez tôt et souvent sur les deux.

Q : On peut se répartir le travail comment ? Comme vous voulez — par écran, par feature, par couche (UI vs logique métier). L'important est que chaque membre ait une contribution identifiable dans l'historique Git et qu'il soit capable de l'expliquer en soutenance.

Q : Le document individuel doit faire combien de pages exactement ? 1 à 2 pages, pas plus. L'objectif est la précision et la honnêteté, pas le volume. Un document de 1 page bien écrit vaut mieux que 3 pages de remplissage.

Q : Que se passe-t-il si l'équipe n'a pas fini toutes les fonctionnalités ? Les fonctionnalités partiellement implémentées valent 0,25 pt. Mieux vaut quelque chose qui fonctionne à moitié qu'un écran vide. Documentez dans le README ce qui est fait et ce qui ne l'est pas.

Bonne chance à toutes et à tous ! 🚀 Vous avez 2 mois et une équipe — c'est exactement les conditions du monde professionnel. Ce que vous construisez maintenant, c'est aussi votre portfolio de demain.